



Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 1

Name : Yashodhan Prakash Suryawanshi
PRN No. : 125B2F004

Class : T.Y. IT - A
Subject : Design Analysis and Algorithm

Github Link: <https://github.com/Yasu225153/DAA-Lab-Submissions/tree/main/Assignment%20-%201>

1. Problem Statement

Design and implement a sorting algorithm using Merge Sort to efficiently arrange customer orders based on their timestamps. The solution should handle a large dataset, up to 1 million orders, with minimal computational overhead. Additionally, analyze the time complexity and compare it with traditional sorting techniques.

2. Theory

Merge Sort is an efficient and stable sorting algorithm based on the **Divide and Conquer** technique. It is widely used for sorting large datasets because it provides a consistent time complexity of $O(n \log n)$. Merge Sort works by repeatedly dividing the input array into smaller sub-arrays, sorting them, and then combining them to produce the final sorted array.

● Working of Merge Sort

The working of Merge Sort can be explained in the following three steps:

1. **Divide:**

The given array is repeatedly divided into two approximately equal halves until each sub-array contains only one element. A single element is considered to be already sorted.

2. **Conquer:**

Each of the smaller sub-arrays is sorted recursively using the same Merge Sort procedure. The division continues until all individual elements are obtained.

3. Merge:

The sorted sub-arrays are then merged by comparing their elements one by one. The smaller element is placed first, and the process continues until all elements are combined into a single sorted array.

4. Final Sorted Array:

The merging process continues at each level of recursion until the complete array is obtained in the required order. In the given application, the records are merged according to their **timestamp values in increasing order**.

3. Complexity Analysis

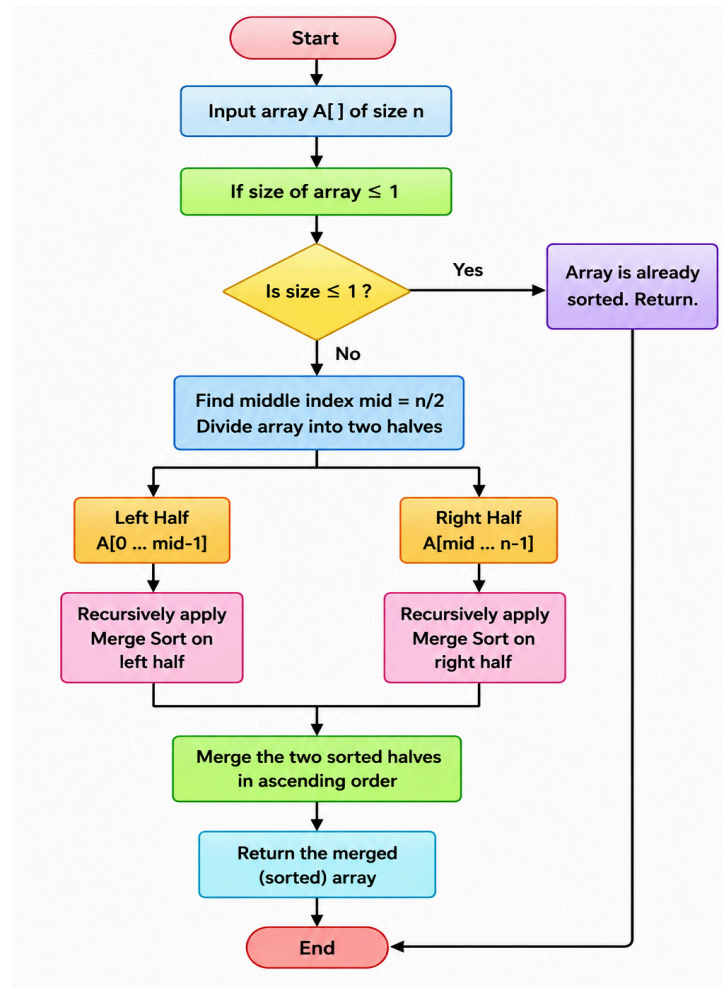
At each recursion level, the array is split into two parts. There are approximately $\log_2(n)$ recursion levels, and n elements are processed during merging at each level. Hence, the overall time complexity is $O(n \log n)$.

Case	Time Complexity	Reason
Best Case	$O(n \log n)$	The array is still divided and merged.
Average Case	$O(n \log n)$	Merging requires work proportional to n at each level.
Worst Case	$O(n \log n)$	The divide-and-merge process remains the same.
Space	$O(n)$	An auxiliary array is used during merging.

4. Merge Sort Algorithm

1. Check if the array has zero or one element; if yes, it is already sorted.
2. Find the middle position of the array.
3. Divide the array into two halves.
4. Recursively apply Merge Sort to both halves.
5. Compare the elements of the two sorted halves.
6. Merge them in ascending order.
7. Continue merging until the complete array is sorted.

5. Flow Diagram



6. Pseudo code

```
// MERGE SORT ALGORITHM
Algorithm MergeSort(arr, low, high)
```

```
    if low >= high then
        return
    end if
```

```
    mid ← (low + high) / 2
```

```
    // Sort left half
    MergeSort(arr, low, mid)
```

```
    // Sort right half
    MergeSort(arr, mid + 1, high)
```

```
    // Merge sorted halves
    Merge(arr, low, mid, high)
```

```
End Algorithm
```

Algorithm Merge(arr, low, mid, high)

Create empty array temp

left $\leftarrow$ low

right $\leftarrow$ mid + 1

while left $\leq$ mid AND right $\leq$ high **do**

if arr[left] $\leq$ arr[right] **then**

 Add arr[left] to temp

 left $\leftarrow$ left + 1

else

 Add arr[right] to temp

 right $\leftarrow$ right + 1

end if

end while

while left $\leq$ mid **do**

 Add arr[left] to temp

 left $\leftarrow$ left + 1

end while

while right $\leq$ high **do**

 Add arr[right] to temp

 right $\leftarrow$ right + 1

end while

for i $\leftarrow$ low to high **do**

 arr[i] $\leftarrow$ temp[i - low]

end for

End Algorithm

7. C++ Program

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:

    // Function to merge two sorted halves
    void merge(vector<string>& orders, int low, int mid, int high) {

        vector<string> temp;
```

```

    int left = low;
    int right = mid + 1;

    // Compare timestamps and store smaller one
    while (left <= mid && right <= high) {

        if (orders[left] <= orders[right])
            temp.push_back(orders[left++]);
        else
            temp.push_back(orders[right++]);
    }

    // Copy remaining elements from left half
    while (left <= mid)
        temp.push_back(orders[left++]);

    // Copy remaining elements from right half
    while (right <= high)
        temp.push_back(orders[right++]);

    // Copy sorted elements back to original array
    for (int i = low; i <= high; i++)
        orders[i] = temp[i - low];
}

// Recursive Merge Sort function
void mergeSort(vector<string>& orders, int low, int high) {

    if (low >= high)
        return;

    // Find middle position
    int mid = low + (high - low) / 2;

    // Sort left half
    mergeSort(orders, low, mid);

    // Sort right half
    mergeSort(orders, mid + 1, high);

    // Merge both sorted halves
    merge(orders, low, mid, high);
}

};

int main() {

    int n;

    cout << "Enter number of customer orders: ";
    cin >> n;

    vector<string> orders(n);

```

```

    cout << "Enter order timestamps (HH:MM):" << endl;

    for (int i = 0; i < n; i++) {
        cin >> orders[i];
    }

    Solution sol;

    // Apply Merge Sort
    sol.mergeSort(orders, 0, n - 1);

    cout << "\nSorted customer orders by timestamp:" << endl;

    for (int i = 0; i < n; i++) {
        cout << orders[i] << " ";
    }

    cout << endl;

    return 0;
}

```

8. Sample Input and Output

Input:

Enter number of customer orders: 4

Enter order timestamps (HH:MM):

12:30

09:15

14:45

10:00

Output:

Sorted customer orders by timestamp:

09:15 10:00 12:30 14:45

9. Step-by-Step Example

Input: [12:30, 09:15, 14:45, 10:00]

Divide: Split the array into [12:30, 09:15] and [14:45, 10:00]. Continue dividing until each part contains one element: [12:30], [09:15], [14:45], [10:00].

Merge: Merge [12:30] and [09:15] → [09:15, 12:30]. Merge [14:45] and [10:00] → [10:00, 14:45].

Final Merge: Combine the two sorted halves → [09:15, 10:00, 12:30, 14:45].

Output: Orders sorted by timestamp → [09:15, 10:00, 12:30, 14:45].

10. Comparison with Traditional Sorting Techniques

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

For a large dataset containing up to **1 million customer orders**, Merge Sort is more efficient than Bubble Sort, Insertion Sort, and Selection Sort because it maintains **$O(n \log n)$** complexity even in the worst case. Its stability also ensures that orders having the same timestamp maintain their original order.

11. Advantages of Merge Sort

- Provides a guaranteed **$O(n \log n)$** time complexity.
- Efficient for handling large datasets.
- Maintains the relative order of equal elements.
- Uses the **Divide and Conquer** approach.
- Provides consistent performance in all cases.

12. Limitations of Merge Sort

- Requires **$O(n)$** additional memory for merging.
- Recursive operations introduce some overhead.
- For small datasets, simpler algorithms such as Insertion Sort may be faster in practice.

13. Result

The Merge Sort algorithm was successfully implemented in C++ to sort customer order timestamps in ascending order. The program divides the data into smaller sections, sorts them recursively, and merges the sorted sections to obtain the final result.

14. Conclusion

Merge Sort is an efficient and stable algorithm for sorting customer orders according to their timestamps. It uses the Divide and Conquer technique and provides $O(n \log n)$ time complexity in all cases. Compared with traditional sorting methods such as Bubble Sort, Insertion Sort, and Selection Sort, it is more suitable for large datasets and practical applications such as e-commerce order processing, order tracking, scheduling, and transaction management.

15. Given C++ Program for CSV Dataset

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
using namespace std;

struct Record {
    string id;
    string name;
    string timestamp;
};

// Merge two sorted parts
void merge(vector<Record>& arr, int left, int mid, int right) {
    vector<Record> temp;

    int i = left;
    int j = mid + 1;

    while (i <= mid && j <= right) {
        if (arr[i].timestamp <= arr[j].timestamp) {
            temp.push_back(arr[i++]);
        } else {
            temp.push_back(arr[j++]);
        }
    }

    while (i <= mid)
        temp.push_back(arr[i++]);

    while (j <= right)
        temp.push_back(arr[j++]);

    for (int k = left; k <= right; k++)
        arr[k] = temp[k - left];
}

// Merge Sort function
void mergeSort(vector<Record>& arr, int left, int right) {
    if (left >= right)
        return;

    int mid = (left + right) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}
```



```
int main() {
    vector<Record> data;

    // Open CSV file
    ifstream file("test.csv");

    string line;

    // Skip the header
    getline(file, line);

    // Read records from CSV
    while (getline(file, line)) {
        stringstream ss(line);
        Record r;

        getline(ss, r.id, ',');
        getline(ss, r.name, ',');
        getline(ss, r.timestamp);

        data.push_back(r);
    }

    file.close();

    // Sort records by timestamp
    mergeSort(data, 0, data.size() - 1);

    cout << "Sorted Data:\n";

    for (const auto& r : data) {
        cout << r.id << " "
              << r.name << " "
              << r.timestamp << endl;
    }

    return 0;
}
```

16. Dataset / Program Execution Screenshot

The following screenshot shows the dataset/output displayed in the terminal while executing the Merge Sort program.



```
yashodhan@yashodhan-ASUS-TUF-Gaming-A17-FA706IHRB-FA706IHRB: ~/Desktop/125B2F004/Assignment - 1$ ./a.out
Number of records read: 10

Sorted Data:
-----
2 Bob 2023-02-10 09:15:20
1 Alice 2023-08-15 16:20:00
8 Helen 2023-12-01 10:40:50
5 Emma 2024-03-12 13:05:25
6 Frank 2024-07-08 15:55:40
4 David 2024-11-22 11:30:45
10 Jack 2025-01-30 08:10:15
7 Grace 2025-06-18 14:25:30
3 Charlie 2025-09-25 19:15:35
9 Ian 2026-01-05 18:45:10
```

Figure 1: Terminal screenshot showing the customer/order dataset.

17. Explanation of the Given Program

- **Record Structure:** The `Record` structure contains three fields: `id`, `name`, and `timestamp`.
- **CSV File:** The program reads customer records from `test.csv`, which should be placed in the same folder as the executable.
- **Reading Data:** `stringstream` splits each CSV row using commas and stores the values in a `Record` object.
- **Merge Function:** The `merge` function compares timestamps from two sorted sections and combines them in ascending order.
- **Merge Sort:** The `mergeSort` function divides the records into smaller halves and recursively sorts them.
- **Output:** After sorting, all customer records are displayed in the terminal according to their timestamps.